



Programação Avançada 2025/26

1 Introdução aos Tipos Abstratos de Dados

@Patricia Macedo & Bruno Silva

Sumário

- Tipos abstratos de dados (ADTs)
 - Acrónimo anglo-saxónico: **Abstract Data Type**
- Metodologia de implementação em *Java*
- ADT Stack

Tipos abstratos de dados | O que são?

Podemos definir tipos abstratos de dados como unidades sintáticas que definem um tipo (de dados) tal que:

- a interface (ou especificação) do tipo **não depende** da (nem obriga a nenhuma) representação concreta dos dados;
- a representação concreta dos dados e a sua manipulação, i.e., a **estrutura de dados**, é oculta do programa que a utiliza.

Tipos abstratos de dados | O que são?

- São tipos de dados que podemos utilizar em programas sabendo apenas a sua especificação e comportamento esperado, ignorando os detalhes da sua implementação.
- Se a implementação mudar, o tipo comporta-se exatamente da mesma forma, sem necessidade de alterar o programa.

ADTs do tipo Coleção

- Nas ciências de computação os tipos abstratos de dados mais "úteis" são conhecidos por ****coleções****, i.e., tipos de dados que permitem armazenar um conjunto de elementos.
 - O tipo de "relação" entre elementos permite classificar o tipo abstrato de dados / coleção, e.g.:
 - **Linear** (stack, queue, list)
 - **Não-linear**
 - **Hierárquico** (árvores)
 - **Não-hierárquico** (grafos)

ADTs do tipo Coleção | Utilidade

- São utilizados para:
 - guardar/representar informação diversa;
 - resolver mais facilmente determinados problemas (abstraindo a gestão interna dos elementos);
 - suportar a implementação de diversos algoritmos.

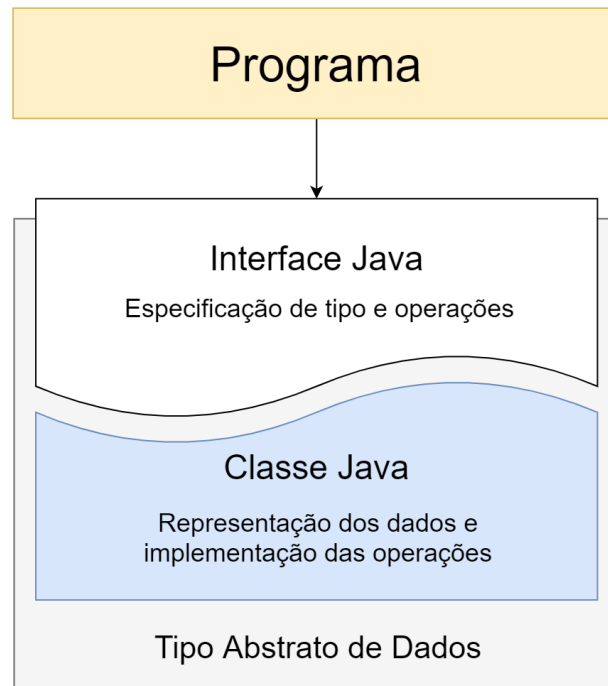
Metodologia ADT C vs Java

Exemplo para ADT Stack:

	C	Java
<i>Especificação</i>	stack.h	interface Stack.java
<i>Tipo armazenado</i>	StackElem	Tipo genérico <T>
<i>Controlo de erros</i>	retorno numérico	Excepções Java
<i>Implementação</i>	stackArrayList.c	StackArrayList.java

- A escolha da estrutura de dados é sempre "escondida" na implementação.
- Podem existir múltiplas implementações, sendo que em Java é possível escolher a mesma através de instanciação.

Metodologia ADT em Java



ADTs (coleções) em Java

A *Java Collections Framework* contém um conjunto de interfaces e classes no *package* `java.util`; as coleções são especificadas através de três interfaces principais:

Interfaces	Implementações
List	ArrayList, LinkedList
Set	HashSet, TreeSet
Map	HashMap, TreeMap, Hashtable

Link: https://en.wikipedia.org/wiki/Java_collections_framework

- As implementações variam nas estruturas de dados utilizadas.
- Obedecem à metodologia apresentada.

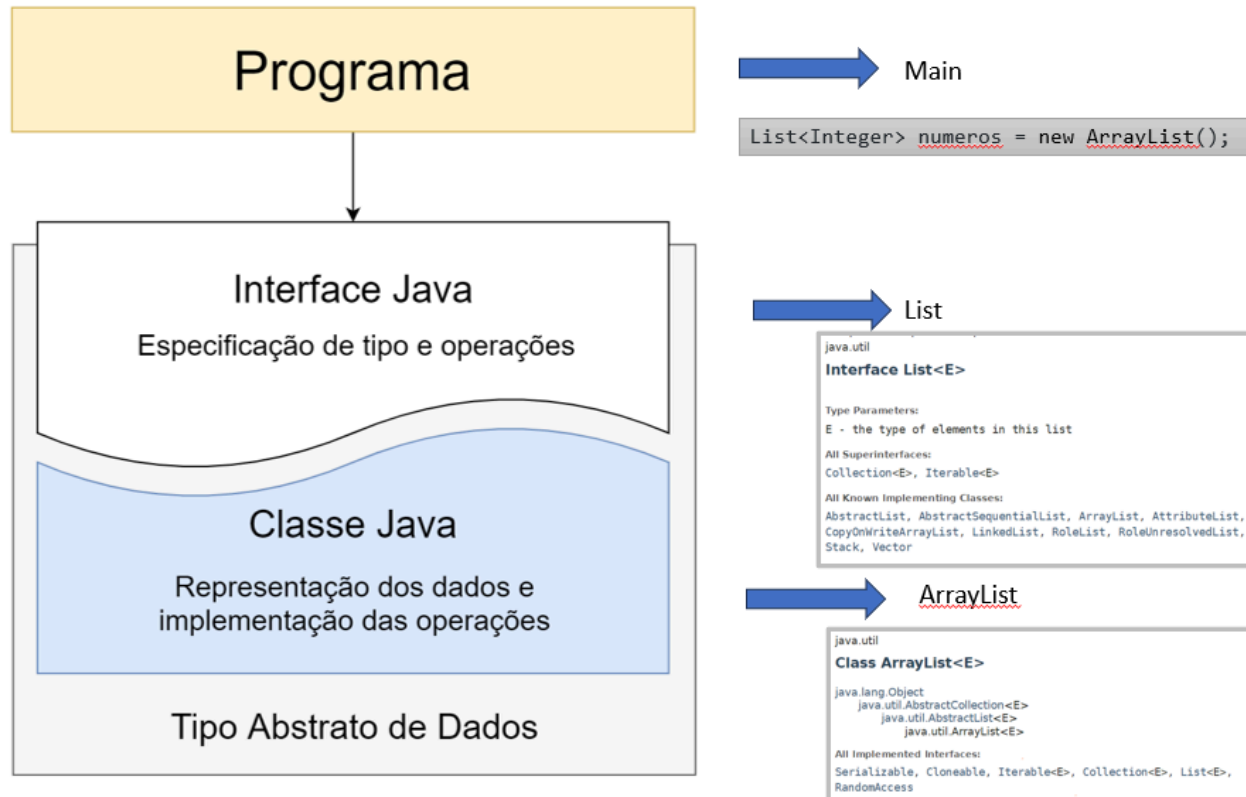
ADTs (coleções) em Java | Exemplo

```
public class Main{  
    public static int main(String[] args){  
        List<Integer> numeros = new ArrayList();  
        for (int i = 1; i <= 50; i++) numeros.add(i);  
        Collections.shuffle(numeros);  
        List<Integer> sorteio = numeros.subList(0, 5);  
        int suplementar = numeros.get(6);  
        Collections.sort(sorteio);  
        System.out.println(sorteio + " + " + suplementar);  
    }  
}
```

QUAL É O RESULTADO? QUAL O PREPÓSITO?

Note a utilização do tipo genérico <Integer> para definir o tipo de elementos a armazenar; será verificado pelo compilador.

Metodologia ADT em Java aplicada ao exemplo anterior



ADT Stack | Desafio - Conversor Decimal Binário

Um programa deverá solicitar um número inteiro ao utilizador e apresentar esse número em binário. O algoritmo divide sucessivamente o número por 2 até zero e guarda numa pilha o resto das divisões. O número em binário é obtido removendo todos os elementos da pilha.

Decimal: 233

Binary: 11101001

$233 // 2 = 116 \mid 233 \% 2 = 1$

$166 // 2 = 83 \mid 166 \% 2 = 0$

$83 // 2 = 41 \mid 83 \% 2 = 1$

$41 // 2 = 20 \mid 41 \% 2 = 1$

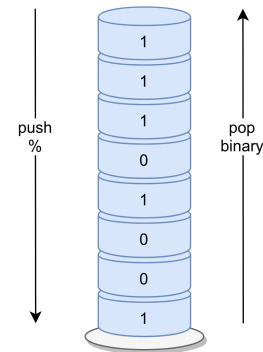
$20 // 2 = 10 \mid 20 \% 2 = 0$

$10 // 2 = 5 \mid 10 \% 2 = 0$

$5 // 2 = 2 \mid 5 \% 2 = 1$

$2 // 2 = 1 \mid 2 \% 2 = 0$

$1 // 2 = 0 \mid 1 \% 2 = 1$



Exemplo de metodologia: ADT Stack

Numa primeira abordagem, poderíamos usar a classe `java.util.Stack` disponibilizada que implementa o ADT java.

```
public static String decimal2Binary(int decimal) throws IllegalArgumentException{
    if( decimal < 0)
        throw new IllegalArgumentException("number should be greater than zero");
    Stack<String> stack= new Stack<>();
    if(decimal==0) return "0";
    while(decimal>0){
        int x= decimal%2;
        decimal=decimal/2;
        stack.push(x==1 ? "1":"0");
    }
    String str="";
    while(!stack.isEmpty()){
        str=str.concat(stack.pop());
    }
    return str;
}
```

Exemplo de metodologia: ADT Stack

Vamos agora imaginar que não temos o ADT Stack implementado em JAVA e vamos fazer a nossa implementação, de forma a estudarmos a implementação de coleções na linguagem Java.

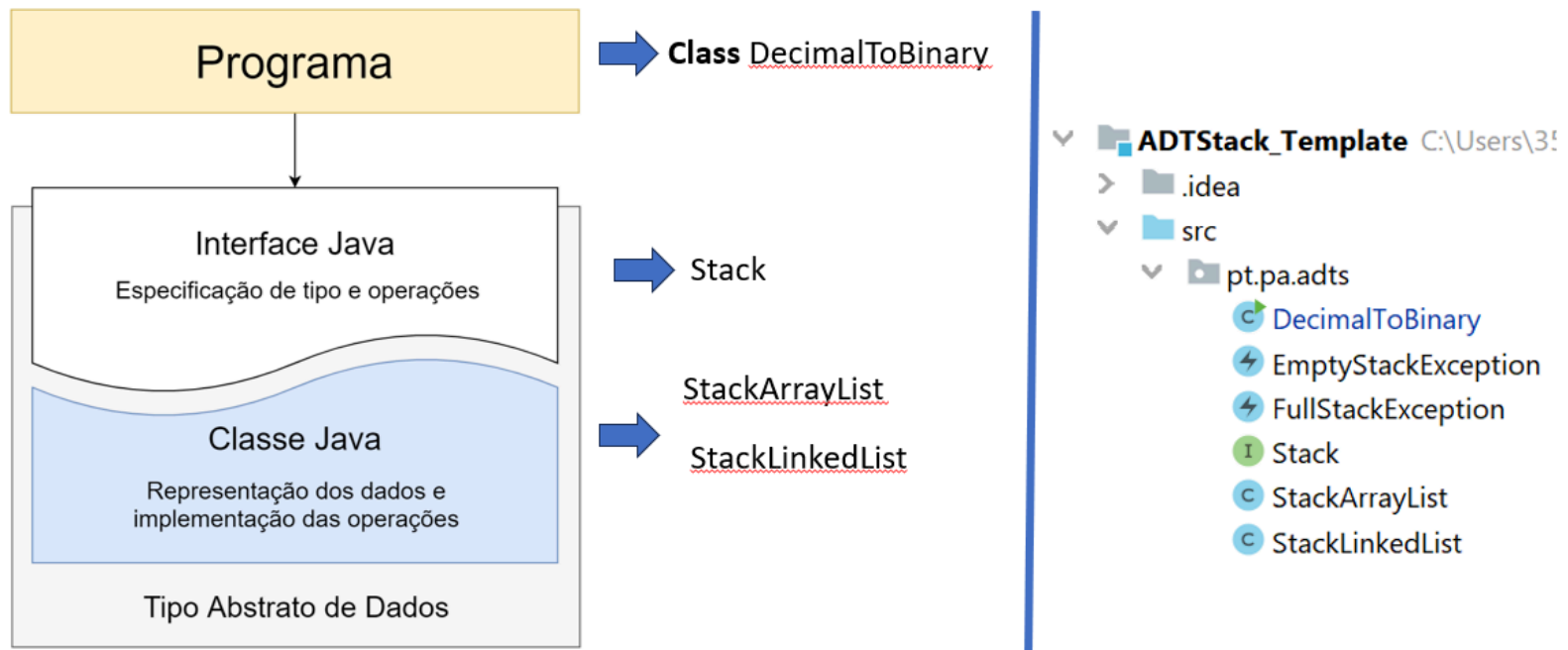
Como tal, vamos incluir as nossas coleções no package:

```
pt.pa.adts
```

Sempre que uma interface/classe tiver o mesmo nome que alguma da Java Collections Framework, a desambiguação será feita pelo nome do package!

ADT Stack | Desafio de aplicação

- Arquitetura para resolução do desafio usando a metodologia dos ADTs



Desafio ADT Stack |Exer. de Implementação

1. IMPLEMENTAR O ADT STACK

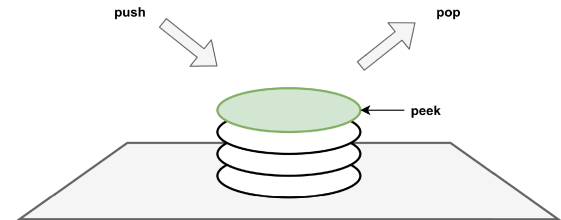
- Definir a Interface Stack
- Implementar as classes que implementam a interface Stack

2. IMPLEMENTAR/ADAPTAR O PROGRAMA

- DecimalToBinary

(1) ADT Stack Rever a Política de acesso LIFO

- Podemos inserir elementos em qualquer altura, mas só podemos aceder/retirar (a) o elemento mais recente na coleção. ➡ i.e., o elemento no "topo" da pilha.
- LIFO: *last in, first out*:
 - significa que o último elemento a entrar, é o primeiro a sair.
 - analogia a uma pilha de pratos, onde podemos ir empilhando pratos, mas só podemos remover o prato que está no **topo**.



===

As operações fundamentais sobre uma *stack* **S** são:

- **push(e)** - insere o elemento **e** no topo de **S**; devolve *erro* se não houver capacidade/memória para mais elementos.
- **pop()** - remove e devolve o elemento no topo de **S**; devolve *erro* se vazia;
- **peek()** - devolve, sem remover, o elemento no topo de **S**; devolve *erro* se vazia;

Operações genéricas de coleções:

- **size()** - devolve o tamanho (nº de elementos) atualmente em **S**;
- **isEmpty()** - devolve um valor lógico (true/false) que indica se **S** está vazia, ou não;
- **clear()** - descarta todos os elementos de **S** voltando a estar vazia;

(1) ADT Stack | Implementação em Java

Especificação do "comportamento" do ADT na interface `Stack.java` (inclui tipo genérico e erros possíveis):

```
public interface Stack<T> {  
    public void push(T element) throws FullStackException;  
    public T pop() throws EmptyStackException;  
    public T peek() throws EmptyStackException;  
  
    public int size();  
    public boolean isEmpty();  
    public void clear();  
}
```

O tipo genérico `<T>` representa qualquer tipo *referenciado* (não-primitivo) na linguagem, que deriva obrigatoriamente da classe `Object`, sendo encarado como desse tipo.

(1) ADT Stack | Implementação em Java

Definição das exceções relativas aos erros possíveis:

```
public class EmptyStackException extends RuntimeException {  
    public EmptyStackException(String message) {  
        super(message);  
    }  
  
    public EmptyStackException() {  
        super("The stack is empty.");  
    }  
}  
  
public class FullStackException extends RuntimeException {  
    public FullStackException(String message) {  
        super(message);  
    }  
  
    public FullStackException() {  
        super("The stack is full.");  
    }  
}
```

(1) ADT Stack|Implementação em Java

Através de uma classe que implemente a interface `Stack.java` ; o nome deverá aludir à estrutura de dados utilizada, e.g.:

```
public class StackArrayList<T> implements Stack<T> {  
    //representação da informação, i.e. estrutura de dados  
    private T[] elements;  
    private int size;  
  
    public StackArrayList() {  
        //inicialização dos atributos -> pilha vazia  
        //...  
    }  
  
    @Override  
    public void push(T element) throws FullStackException {  
        //...  
    }  
  
    //implementação dos restantes métodos da interface  
}
```

Exercícios Práticos

ADT Stack | Exer. de implementação

1. Faça *clone* do projeto base **ADTStack_Template** (projeto **IntelliJ**) do *GitHub*:

https://github.com/estsetubal-pa/ADTStack_Template

2. Forneça a documentação para a classe `StackArrayList`, sabendo que utiliza uma estrutura de dados baseada em *array* (da classe e atributos; a documentação dos métodos da interface é herdada).

Exercícios Práticos (cont)

3. Forneça o código dos métodos por implementar, i.e., os que estão a lançar `UnsupportedOperationException` ;
4. Compile e teste o programa fornecido, verificando que os resultados são os esperados; a exceção `FullStackException` deverá ser capturada com sucesso.
5. Modifique o método `push()` para aumentar dinamicamente o array `elements` sempre que necessário. Compile e teste novamente o programa; a exceção já não deverá ocorrer.

Exercícios Práticos (cont)

6. Pretende-se uma diferente implementação baseada em *lista (simplesmente) ligada* na classe `StackLinkedList`. A definição de um nó é fornecida na *inner class* `Node`.

```
public class StackLinkedList<T> implements Stack<T> {  
    private Node top; //sentinela  
    private int size;  
  
    public StackLinkedList() {  
        this.top = new Node(null, null);  
        this.size = 0;  
    }  
    private class Node { //inner class, só reconhecida neste contexto  
        private T element;  
        private Node next;  
        public Node(T element, Node next) {  
            this.element = element;  
            this.next = next;  
        }  
    }  
}
```


Exercícios Práticos (cont)

8. Substitua a implementação de `Stack` utilizada no método `main()` por uma instância da classe anterior. Compile e teste o programa verificando que o comportamento do programa se mantém inalterado.
9. Defina a classe `DecimalToBinary` e implemente o método `decimal2binary` (ver solução fornecida), agora usando a classe `StackArrayList`, em vez da `java.util.Stack`.

Bibliografia



pp 40.

António Adrego da Rocha, **Estruturas de Dados e Algoritmos em Java**, 2011. ISBN- 978-972-722-704-4, FCA